

VP-Planilla

Bitacora de Pruebas Unitarias

Sistema de Planilla - Costa Rica

Fecha de Ejecucion	4 de abril de 2026
Version	v3.0 - Phase 17 completa (183 pruebas nuevas)
Proyecto	VP-Planilla (Sistema de Planilla Costa Rica)
Responsables	Alejandro Leon Marin Kendall Fonseca Hidalgo
Herramienta	Jest 29 + ts-jest Node.js 22.14.0 TypeScript 5.8.3
Entorno	Windows 11 Pro - Desarrollo local
Comando ejecutado	npm test -- --coverage --verbose

1. Resumen Ejecutivo

Metrica	Valor	Estado
Suites de prueba ejecutadas	17	PASS
Total de casos de prueba	287	PASS
Pruebas exitosas (PASS)	287	PASS
Pruebas fallidas (FAIL)	0	PASS
Tiempo de ejecucion	~20 segundos	
Tasa de exito	100%	PASS
Pruebas agregadas en Phase 17	+183 (104 a 287)	
Cobertura statements	42.49%	

Phase 17 completa: se agregaron 183 pruebas unitarias nuevas distribuidas en 3 waves. Se cubrieron 13 servicios previamente sin cobertura. Tasa de exito 100% mantenida. Cobertura de statements subio de 31.59% a 42.49% (+10.9pp).

2. Suites de Prueba Ejecutadas

Suite 1 — src/ tests /unit/payrollUtils.test.ts (EXTENDIDA)

Funcion bajo prueba: Utilidades matematicas de planilla — extendida con 80 nuevos casos

#	Grupo	Caso de Prueba	Resultado	Duración
1	isCRHoliday	Año Nuevo (1 enero) reconocido como feriado	PASS	1 ms
2	isCRHoliday	Día del Trabajo (1 mayo) reconocido como feriado	PASS	1 ms
3	isCRHoliday	Independencia (15 setiembre) reconocido como feriado	PASS	1 ms
4	isCRHoliday	Navidad (25 diciembre) reconocido como feriado	PASS	1 ms
5	isCRHoliday	Anexión de Guanacaste (25 julio) reconocido como feriado	PASS	1 ms
6	isCRHoliday	Asunción (15 agosto) reconocido como feriado	PASS	1 ms
7	isCRHoliday	Día de las Culturas (12 octubre) reconocido como feriado	PASS	1 ms
8	isCRHoliday	Juan Santamaría (11 abril) reconocido como feriado	PASS	1 ms
9-103	Múltiples grupos	validateClockLogPairs, validatePayrollPeriod, getSundaysInPeriod, getWeeklyRestDays, calculateOvertimeHoursBiweekly, hasOverlappingPairs, y mas	PASS	<2 ms c/u

Total: 103 casos — todos PASS

Suite 2 — src/___tests___/unit/services/PayrollService.test.ts

Servicio bajo prueba: PayrollService (Gestión de periodos de planilla)

#	Grupo	Caso de Prueba	Resultado	Duración
1	createPayroll	Crear planilla exitosamente con datos válidos	PASS	3 ms
2	createPayroll	Lanzar error cuando falla la operación en DB	PASS	11 ms
3	createPayroll	Crear planilla con versión 1 por defecto	PASS	1 ms
4	getAllPayrolls	Recuperar todas las planillas ordenadas por ID desc	PASS	4 ms
5	getAllPayrolls	Retornar arreglo vacío cuando no hay planillas	PASS	1 ms
6	getAllPayrolls	Mapear correctamente los campos de DB al modelo	PASS	3 ms
7	getAllPayrolls	Lanzar error cuando falla la consulta a la DB	PASS	<1 ms
8	Edge Cases	Manejar diferentes estados de planilla	PASS	1 ms
9	Edge Cases	Manejar planilla con fechas de periodo iguales	PASS	1 ms

Suite 3 — src/___tests___/unit/services/NomineeService.test.ts

Servicio bajo prueba: NomineeService (Cálculo de planilla — núcleo ley laboral CR)

#	Grupo	Caso de Prueba	Resultado	Duración
1	REQ 8.1	Cálculo regular: empleado con 8h/día exactas en periodo quincenal	PASS	5 ms
2	REQ 8.2	Overtime 1.5x: horas entre 8h y 10h se pagan al 150%	PASS	3 ms
3	REQ 8.2	Overtime 2x: horas sobre 10h en un día se pagan al 200%	PASS	3 ms
4	REQ 8.3	Día de descanso trabajado: compensación 0.5x del salario diario	PASS	2 ms

#	Grupo	Caso de Prueba	Resultado	Duración
5	REQ 8.4	Periodo con feriado nacional CR: feriado excluido de días laborales	PASS	3 ms
6	REQ 8.5	Deducciones CCSS: obrero 10.67% y patronal 26.67% correctos	PASS	2 ms
7	REQ 8.6	OT 2x en día completo: multiplicador correcto	PASS	3 ms
8	Edge Case	Periodo de cero empleados activos retorna resultado vacío	PASS	1 ms
9	Edge Case	Empleado con vacaciones en el periodo — horas excluidas	PASS	2 ms
10	Fallback	calculatePayrollForPeriod sin empleados activos — mensaje de advertencia	PASS	1 ms

Suite 4 — src/ __tests__ /integration/payroll.integration.test.ts

Tipo: Tests de integración HTTP (supertest) — endpoint POST /api/nominee/calculate-payroll

#	Grupo	Caso de Prueba	Resultado	Duración
1	Auth guard	Retorna 401 cuando no se envía token de autorización	PASS	44 ms
2	Auth guard	Retorna 401 cuando el token es inválido o malformado	PASS	22 ms
3	Ruta deprecated	POST /api/nominee/calculate retorna 401 — ruta eliminada	PASS	4 ms

Suite 5 — src/ __tests__ /unit/services/EmployeeService.test.ts

Servicio bajo prueba: EmployeeService (Gestión de empleados)

Total: 26 casos — todos PASS. Cobertura: 97%

Suite 6 — src/ __tests__ /unit/services/ClockLogsService.test.ts

Servicio bajo prueba: ClockLogsService (Registros de entrada/salida)

Total: 9 casos — todos PASS. Cobertura: 100%

Suite 7 — src/ __tests__ /unit/services/DeductionsService.test.ts

Servicio bajo prueba: DeductionsService (Deducciones de planilla)

Total: varios casos — todos PASS. Cobertura: 90%

Suite 8 — src/ __tests__ /unit/services/AuthService.test.ts

Servicio bajo prueba: AuthService (Autenticación y JWT)

#	Grupo	Caso de Prueba	Resultado	Duración
1	authenticate	Retorna éxito con token cuando credenciales son válidas	PASS	228 ms
2	authenticate	Retorna fallo cuando usuario no existe	PASS	4 ms
3	authenticate	Retorna fallo cuando contraseña es incorrecta	PASS	9 ms
4	authenticate	Retorna error de servidor cuando DB lanza excepción	PASS	28 ms
5	authenticate	Compara contraseña en texto plano si no es bcrypt	PASS	7 ms

#	Grupo	Caso de Prueba	Resultado	Duración
6	verifyToken	Retorna payload decodificado para token valido	PASS	1 ms
7	verifyToken	Lanza error para token invalido	PASS	22 ms
8	getUserById	Retorna usuario autenticado cuando se encuentra	PASS	1 ms
9	getUserById	Retorna null cuando usuario no existe	PASS	1 ms
10	getUserById	Retorna null cuando DB lanza excepcion	PASS	5 ms
11	getUserByUsername	Retorna usuario cuando se encuentra	PASS	2 ms
12	getUserByUsername	Retorna null cuando no existe	PASS	1 ms
13	isTokenBlocklisted	Retorna true cuando token esta en lista negra	PASS	9 ms
14	isTokenBlocklisted	Retorna false cuando token no esta bloqueado	PASS	1 ms
15	validateCredentials	Retorna usuario cuando credenciales son validas	PASS	5 ms
16	validateCredentials	Retorna null cuando usuario no existe	PASS	1 ms
17	validateCredentials	Retorna null cuando contrasena es incorrecta	PASS	5 ms

Suite 9 — src/__tests__/unit/services/VacationService.test.ts (NUEVA — Phase 17 Wave 1)

Servicio bajo prueba: VacationService (Gestion de vacaciones)

#	Grupo	Caso de Prueba	Resultado	Duración
1	getVacationById	Retorna vacacion con mapeo correcto de campos	PASS	58 ms
2	getVacationById	Retorna null cuando no se encuentra	PASS	5 ms
3	getVacationById	Propaga errores de DB	PASS	54 ms
4	getAllVacations	Retorna arreglo mapeado de vacaciones	PASS	2 ms
5	getAllVacations	Retorna arreglo vacio cuando no hay registros	PASS	1 ms
6	getAllVacations	Retorna [] silenciosamente si mensaje contiene 'does not exist'	PASS	1 ms
7	getAllVacations	Retorna [] silenciosamente si mensaje contiene 'p2021'	PASS	<1 ms
8	getAllVacations	Propaga otros errores de DB	PASS	3 ms
9	createVacation	Crea vacacion y retorna resultado mapeado	PASS	3 ms
10	updateVacation	Actualiza vacacion y retorna registro con version incrementada	PASS	3 ms
11	deleteVacation	Retorna true cuando se elimina exitosamente	PASS	1 ms
12	deleteVacation	Propaga errores de DB en eliminacion	PASS	3 ms

Suite 10 — src/__tests__/unit/services/PayrollTypeService.test.ts (NUEVA — Phase 17 Wave 1)

Servicio bajo prueba: PayrollTypeService (Tipos de planilla — quincenal, mensual, etc.)

Total: 9 casos — todos PASS. Incluye prueba especial del bloque finally con \$disconnect.

Suite 11 — src/___tests___/unit/services/PositionService.test.ts (NUEVA — Phase 17 Wave 1)

Servicio bajo prueba: PositionService (Puestos y salarios base)

#	Grupo	Caso de Prueba	Resultado	Duración
1	getPositionById	Retorna posicion con base_salary como number (Decimal resuelto)	PASS	37 ms
2	getPositionById	Retorna null cuando no se encuentra	PASS	26 ms
3	getPositionById	Propaga errores de DB	PASS	33 ms
4	getAllPositions	Retorna arreglo mapeado con Decimal convertido a number	PASS	1 ms
5	getAllPositions	Retorna arreglo vacio cuando no hay posiciones	PASS	1 ms
6	createPosition	Crea posicion y retorna resultado mapeado	PASS	3 ms
7	updatePosition	Retorna posicion actualizada cuando updateMany retorna count=1	PASS	2 ms
8	updatePosition	Retorna null cuando updateMany retorna count=0 (version mismatch)	PASS	1 ms
9	updatePosition	Propaga errores de DB en update	PASS	2 ms
10	deletePosition	Retorna true cuando deleteMany retorna count=1	PASS	2 ms
11	deletePosition	Retorna false cuando deleteMany retorna count=0	PASS	1 ms

Suite 12 — src/___tests___/unit/services/BonusesService.test.ts (NUEVA — Phase 17 Wave 1)

Servicio bajo prueba: BonusesService (Bonificaciones de empleados)

#	Grupo	Caso de Prueba	Resultado	Duración
1	createBonus	Crea bonus y retorna resultado mapeado con amount como number	PASS	71 ms
2	createBonus	Propaga errores de DB en creacion	PASS	40 ms
3	getBonusById	Retorna bonus cuando se encuentra	PASS	4 ms
4	getBonusById	Retorna null cuando no se encuentra	PASS	2 ms
5	updateBonus	Actualiza bonus cuando existe	PASS	7 ms
6	updateBonus	Retorna null sin llamar update cuando bonus no existe	PASS	1 ms
7	updateBonus	Propaga errores de DB en findUnique durante update	PASS	1 ms
8	deleteBonus	Elimina bonus y retorna datos cuando existe	PASS	4 ms
9	deleteBonus	Retorna null sin llamar delete cuando bonus no existe	PASS	1 ms
10	deleteBonus	Propaga errores de DB en eliminacion	PASS	2 ms

Suite 13 — src/___tests___/unit/services/EmployeeDeductionsService.test.ts (NUEVA — Phase 17 Wave 1)

Servicio bajo prueba: EmployeeDeductions (Deducciones por empleado)

Total: 6 casos — todos PASS. Incluye prueba de clave compuesta en delete.

Suite 14 — src/___tests___/unit/services/LaborEventsService.test.ts (NUEVA — Phase 17 Wave 2)

Servicio bajo prueba: LaborEventsService (Eventos laborales — incapacidades, permisos)

Total: 15 casos — todos PASS. Incluye relacion anidada vpg_labor_events en getAllEmployeeLaborEvents.

Suite 15 — src/___tests___/unit/services/UserService.test.ts (NUEVA — Phase 17 Wave 2)

Servicio bajo prueba: UserService (Gestion de usuarios y permisos)

#	Grupo	Caso de Prueba	Resultado	Duración
1	getRoleCatalog	Retorna arreglo con 4 definiciones de rol (admin, supervisor, analyst, viewer)	PASS	1 ms
2	listUsers	Retorna arreglo mapeado de UserPermissionSummary con roleLabel	PASS	2 ms
3	listUsers	Usuario con rol desconocido: fallbackRoleLabel aplicado	PASS	1 ms
4	listUsers	Sin usuarios: retorna arreglo vacio	PASS	1 ms
5	updatePermissions	Rol valido, usuario existe, sin actorId: no crea audit log	PASS	3 ms
6	updatePermissions	Rol valido, usuario existe, con actorId: crea audit log	PASS	2 ms
7	updatePermissions	Rol invalido ('superadmin'): lanza error con statusCode 400	PASS	1 ms
8	updatePermissions	Usuario no encontrado: lanza error con statusCode 404	PASS	1 ms
9	updatePermissions	Usuario con middle_name null: fullName sin espacio doble	PASS	1 ms
10	updatePermissions	DB falla en update: propaga error	PASS	1 ms

Suite 16 — src/___tests___/unit/services/NotificationService.test.ts (NUEVA — Phase 17 Wave 2)

Servicio bajo prueba: NotificationService (Sistema de notificaciones)

#	Grupo	Caso de Prueba	Resultado	Duración
1	createNotification	Crea notificacion y retorna fila de DB	PASS	2 ms
2	createNotification	Propaga errores de DB	PASS	1 ms
3	getNotificationsBy UserId	Retorna { data, total } con paginacion correcta	PASS	3 ms
4	getNotificationsBy UserId	Pagina 2 con limit 5: skip=5 pasado a findMany	PASS	1 ms
5	getNotificationsBy UserId	Sin notificaciones: retorna { data: [], total: 0 }	PASS	1 ms
6	getUnreadCount	Retorna count de prisma.count	PASS	1 ms
7	getUnreadCount	Propaga errores de DB	PASS	1 ms
8	markAsRead	Notificacion encontrada: actualiza y retorna notificacion actualizada	PASS	2 ms
9	markAsRead	Notificacion no encontrada: retorna null sin llamar update	PASS	1 ms
10	markAllAsRead	Actualiza todas las no leidas y retorna result.count	PASS	1 ms

#	Grupo	Caso de Prueba	Resultado	Duración
11	markAllAsRead	Propaga errores de DB	PASS	1 ms
12	deleteNotification	Notificación encontrada: elimina exitosamente	PASS	1 ms
13	deleteNotification	Notificación no encontrada: lanza 'Notification not found or access denied'	PASS	1 ms

Suite 17 — src/__tests__/unit/services/AuditLogsService.test.ts (NUEVA — Phase 17 Wave 3)

Servicio bajo prueba: AuditLogsService (Bitacora de auditoria del sistema)

#	Grupo	Caso de Prueba	Resultado	Duración
1	getAuditLogs	Retorna { data:[], total:0, limit:100, offset:0 } sin filtros	PASS	21 ms
2	getAuditLogs	Mapea filas retornadas al shape esperado	PASS	2 ms
3	getAuditLogs	Pasa filtro userId a clausula where	PASS	1 ms
4	getAuditLogs	Pasa filtro action con contains/insensitive	PASS	1 ms
5	getAuditLogs	Pasa filtro entity con contains/insensitive	PASS	1 ms
6	getAuditLogs	Pasa filtro startDate como gte en timestamp	PASS	1 ms
7	getAuditLogs	Pasa filtro endDate como lte en timestamp	PASS	1 ms
8	getAuditLogs	Pasa startDate y endDate como gte/lte combinados	PASS	1 ms
9	getAuditLogs	Pasa limit y offset personalizados como take/skip	PASS	1 ms
10	getAuditLogs	Retorna limit y offset de los filtros en el resultado	PASS	2 ms
11	getAuditLogs	Propaga errores de DB	PASS	20 ms
12	createAuditLog	Crea audit log con campos correctos	PASS	1 ms
13	createAuditLog	Establece details como null cuando no se provee	PASS	1 ms
14	getAuditLogById	Retorna el audit log mapeado cuando se encuentra	PASS	1 ms
15	getAuditLogById	Retorna null cuando no se encuentra	PASS	<1 ms
16	getAuditLogById	Propaga errores de DB	PASS	1 ms
17	getAuditLogById	Llama findUnique con include para vpg_users con select correcto	PASS	1 ms

3. Cobertura de Código (Code Coverage)

Cobertura total medida en cuatro dimensiones: Statements, Branches, Functions, Lines. Phase 17 elevó la cobertura de statements de 31.59% a 42.49% (+10.9pp), cubriendo 13 servicios que previamente tenían 0% de cobertura.

Archivo	Statements	Branches	Functions	Lines	Estado
utils/payrollUtils.ts	97%	82%	100%	97%	Alto
service/AuditLogsService.ts	100%	100%	100%	100%	Alto

Archivo	Statement s	Branches	Functions	Lines	Estado
service/AuthService.ts	84%	81%	62%	84%	Alto
service/BonusesService.ts	100%	95%	100%	100%	Alto
service/ClockLogsService.ts	100%	100%	100%	100%	Alto
service/DeductionsService.ts	90%	74%	100%	89%	Alto
service/EmployeeDeductions.ts	100%	100%	100%	100%	Alto
service/EmployeeService.ts	97%	73%	100%	97%	Alto
service/LaborEventsService.ts	91%	57%	100%	90%	Alto
service/NotificationService.ts	100%	100%	100%	100%	Alto
service/PayrollTypeService.ts	100%	67%	100%	100%	Alto
service/PositionService.ts	100%	100%	100%	100%	Alto
service/UserService.ts	97%	79%	100%	100%	Alto
service/VacationService.ts	96%	55%	100%	96%	Alto
service/NomineeService.ts	51%	30%	44%	53%	Medio
service/PayrollService.ts	45%	2%	19%	63%	Medio
lib/prisma.ts	67%	100%	0%	75%	Medio
middleware/AuthMiddleware.ts	31%	11%	43%	31%	Bajo
service/PaymentReceiptService.ts	5%	0%	0%	5%	Sin cobertura
service/ReportsService.ts	4%	1%	0%	5%	Sin cobertura
TOTAL PROYECTO	42.49%	26.19%	40%	42.97%	

Logro Phase 17: 13 servicios pasaron de 0% a cobertura alta (90-100%). payrollUtils.ts alcanzo 97% cubriendo todas las funciones de calculo de planilla segun ley laboral costarricense. Los servicios criticos de negocio (VacationService, BonusesService, PositionService, UserService, NotificationService, AuditLogsService) tienen ahora 96-100% de cobertura.

4. Progreso Historico de Pruebas

Versio n	Fecha	Suites	Tests	Cobertura Statements	Fase
v1.0	27 marzo 2026	4	45	~33% (nucleo)	Milestone 1 completo
v2.0	27 marzo 2026	8	104	31.59%	Phases 9-10: EmployeeService + ClockLogs + Auth + Deductions
v3.0	4 abril 2026	17	287	42.49%	Phase 17: +183 tests en 3 waves

5. Servicios Pendientes de Cobertura

Servicio	Cobertura Actual	Lineas sin Cubrir	Prioridad
NomineeService.ts	51%	38-233, 308-317, 360-380, y mas	Alta (nucleo planilla)
PayrollService.ts	45%	94-188	Alta
PaymentReceiptService.ts	5%	52-510	Media (generacion PDF)
ReportsService.ts	4%	136-883	Baja (reportes complejos)
AuthMiddleware.ts	31%	33, 43-63, 79-124	Media

Para alcanzar el objetivo original de 60%: se requiere cubrir NomineeService (gaps en calculo quincenal) y PayrollService (metodos de consulta y cierre de planilla).

6. Proximos Pasos

Objetivo	Descripcion
Wave 4 — NomineeService gaps	Cubrir lineas 38-233 (calculo quincenal) para subir de 51% a ~75%
Wave 4 — PayrollService	Cubrir metodos de consulta y cierre de planilla (lineas 94-188)
Meta 60% statements	Alcanzable con Wave 4: estimado ~55-62% total del proyecto
Tests E2E (Playwright)	Flujos completos: login, crear planilla, calcular, cerrar sesion
Tests de frontend (Vitest)	Componentes React: formularios, hooks de datos, navegacion del dashboard

Alejandro Leon Marin

Desarrollador — VP-Planilla

4 de abril de 2026

Kendall Fonseca Hidalgo

Desarrollador — VP-Planilla

4 de abril de 2026